

First Steps in R

MASH

2026-09-16

Contents

Chapter 1

About

What this book is about

Chapter 2

Everything you need to know before you learn R

If you are completely new to R, you are in the right place.

Learning R can feel intimidating at first. Unlike software such as Excel and SPSS, R asks you to **type instructions** to tell the computer what you want it to do rather than clicking icons and menus.

That can feel strange at first.

The good news is that you do **not** need to be a programmer to learn R. You just need to learn a few basic ideas, practise them, and gradually build up your understanding.

This document is not intended to teach you R - rather, it should help you get started and highlight the key concepts to know before you try to learn R.

By the end, you should understand:

- What R is
- The difference between R and Rstudio
- Some advantages and disadvantages of R versus other software
- Main parts of the RStudio interface
- How to begin coding in R
- Where to find help and learning materials
- How to continue learning R independently

The most important thing to remember: You do not need to understand everything before you start. Learning R is a process.

2.1 What is R?

R is a **programming language**. That means that instead of telling R what to do by clicking buttons in a menu, you generally type instructions.

For example, you can ask R to calculate 2+2

```
2 + 2
```

and it will return:

```
#> [1] 4
```

R is widely used for statistics, data analysis and data visualisation. It is used by students, researchers, universities, governments, businesses and many other organisations.

R is especially useful when you need to:

- Work with data
- Perform calculations
- Carry out statistical analyses
- Create graphs and visualisations
- Manipulate datasets

2.1.1 What is RStudio?

You will often hear people talk about **R** and **RStudio** as though they are the same thing.

They are not.

R is the programming language itself. It is the thing that actually performs your calculations, analyses and commands.

RStudio is an Integrated Development Environment (IDE) for R. A fancy term for a user dashboard. It provides a convenient interface for working with R.

Think of it this way: If R is the engine, RStudio is the dashboard that makes the engine easier to use.

You can use R without RStudio, but RStudio provides a much more convenient environment for writing, running and organising R code.

When you are getting started, you will normally install **both R and RStudio**.

2.1.2 Why learn R?

There are lots of reasons to learn R.

2.1.2.1 It's free!

One of the biggest advantages of R is that it is **free**. You do not need to buy a licence to continue using R after you graduate and there are no limits to how many devices you install it on.

2.1.2.2 It's flexible!

R can be used for an enormous range of tasks from calculating statistics, plotting data, statistical analysis, hypothesis tests, regression, automating tasks, writing reports and much more.

2.1.2.3 It's widely used!

R is widely used in scientific research and academia, as well as industry. R has an enormous library of **packages** to help you perform even quite specific tasks.

Packages are collections of additional functions, data and other resources that extend what R can do.

R can do a huge amount. And if it cannot already do something you need, there is often a package that can help — or you can potentially create your own solution.

2.1.3 The downsides of R

R is powerful, but there are some things that can make it difficult to use when you first start. The main thing being that you have to interact with it as a programming language.

If you are used to software where you select options from menus, R may initially feel unfamiliar and there will initially be quite a lot to learn.

Before being proficient in R, you will need to learn:

- How R understands commands
- What objects are
- How functions work
- How to structure your code
- How to find and fix errors

This can feel daunting at first but is **** completely normal.****

The first few sessions of learning R are often the hardest because you are learning a new way of communicating with a computer.

In software such as Excel, to plot a graph you might:

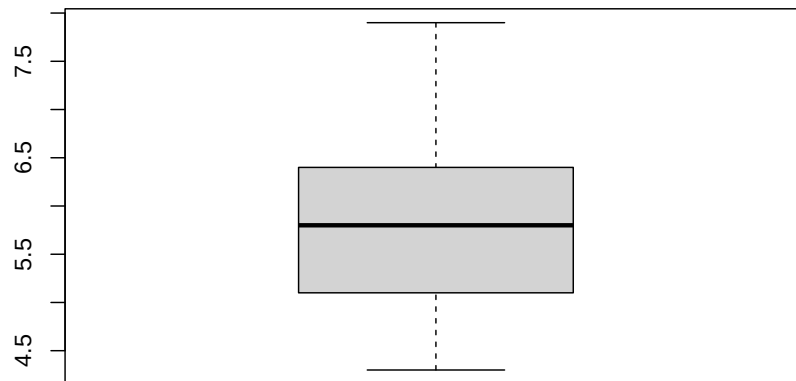
1. select some cells;
2. click *insert*;
3. select the chart type you want;

4. click OK.

In R, you might instead type something like:

```
boxplot(my_data)
```

and get a plot such as



2.2 Installing R and RStudio

The University of Sheffield MASH R resources page contains a guide to downloading and setting up both R and RStudio.

Alternatively, go to both the R and RStudio webpages and download the appropriate version for your device.

2.3 Getting to know RStudio

When you open RStudio, you will probably see several different sections or **panes**.

The exact arrangement can vary depending on your version and settings.

A common layout contains four main panes:

1. (Top-left) **Source**
2. (Bottom-left) **Console**

3. (Top-right) **Environment** / **History**
4. (Bottom-left) **Files** / **Plots** / **Packages** / **Help**

2.3.1 1. Source

The **Source** pane is where you can write, edit and save your code. If it is the first time opening Rstudio, the **Source** pane may not appear until you click the following.

File > New File > R Script

The **Source pane** is where you should normally develop your analysis. The saved code is referred to as a script and has the file extension `.R`. These files can be reopened and worked on at any time.

2.3.2 2. Console

The **Console** is where R receives and executes commands.

You can type directly into it. The result will appear underneath. Unlike an R script, the **Console** is temporary and anything you type into it is not saved. The **Console** is particularly useful for:

- Trying out commands
- Testing ideas
- Checking the result of a function
- Troubleshooting

Tip: you can press the **Up** key with your cursor in the Console to quickly cycle through commands you have typed recently**

2.3.3 3. Environment / History

The **Environment** pane shows objects that currently exist in your R session.

For example, if you run the code:

```
student_scores <- c(65, 72, 81, 69, 88)
```

the object `student_scores` will appear in the **Environment**. *We'll talk more about objects later.*

The **Environment** therefore gives you a useful overview of the objects you have created.

There are other tabs within this pane, the next most useful being the **History** tab which shows all the commands that you have previously entered.

This can be useful when you are trying to remember something you did earlier in your session.

2.3.4 4. Files / Plots / Packages / Help

The last window contains several tabs including **Files**, **Plots**, **Packages** and **Help**.

The **Files** tab allows you to navigate through folders on your computer (see **Working Directories** for more detail).

Plots displays graphs that you create in R.

Packages allows you to see packages that are installed in your R session.

Help displays R's documentation when you request it using the `?` command (see Errors for further help).

The following table summarises the pane functions.

	Window	What it does
Source		Write and edit R scripts
Console		Run R commands
Environment		View objects currently stored in R
History		View previously entered commands
Files		Navigate through files and folders
Plots		View graphs
Packages		Install and manage packages
Help		Access R documentation

2.3.5 Working directories

One concept that often causes confusion when you first start using R is the **working directory**.

The **working directory** is the folder that R is currently using as its default location for reading and writing files.

You can find your current working directory by running:

```
getwd()
```

`getwd()` means **get working directory**.

You can change the working directory using `setwd()` and the path to your desired folder. For example:

```
setwd("C:/Users/YourName/Documents/R")
```

The exact path will depend on your computer.

Tip: You do not necessarily need to use `setwd()`. Instead, in the **Files** tab, click `...` on the right-hand side, click through the file ex-

plorer to find the folder you want then click **Open**. Click the cog symbol titled **More** then Select **Set as Working Directory**.

2.4 Learning how to write code

There are a few things that are worth understanding right at the beginning.

R is primarily controlled by typing commands. You can think of each line of R code as an instruction.

For example, to find the mean of the `student_scores` given above you'd type:

```
mean(student_scores)
```

It is very important before learning R to realise that **you do not have to memorise everything**.

One of the most important skills in R is knowing how to find information. It is unlikely that you will remember the code for everything you want to do.

That is normal.

Learning R is not about memorising an enormous list of commands. It is about learning enough of the language that you can understand, debug (fix) and build code.

The University of Sheffield MASH R resources page provides first steps in R.

These first steps will guide you through:

- Installing R
- Downloading RStudio
- Interacting with R
- Understanding objects and functions
- Working with vectors
- Getting started with data
- Installing packages

These are a good place to get started!

2.4.1 What is a command?

A **command** is an instruction that you give to R.

To run a command in the **Console** you simply type your command and press **Enter**.

To run a command in the **Source** pane (script) there are two ways:

- Highlight the line(s) of code and press **Ctrl + Enter** on your keyboard or the **Run** key in the top right of the Source pane.

- Put your cursor on the line of code and press **Ctrl + Enter**. This also moves the cursor to the next line so you can easily run the next line.

Tip: To quickly highlight your entire script click **Ctrl + A** with your cursor anywhere in the script.

An example command is:

```
2 + 2
```

an instruction asking R to add two numbers.

Another example is:

```
even_numbers <- c(2, 4, 6, 8)
```

This tells R to create an *object* named `even_numbers` by combining the numbers 2, 4, 6 and 8. Note here, that in programming we call this a vector. Vectors are formed in R using the function `c()` where `c` stands for combine.

You will quickly become familiar with the basic structure of R commands.

2.4.2 What is a function?

A **function** is a piece of code designed to perform a particular task.

Functions are one of the most important concepts in R.

For example:

```
mean(c(2, 4, 6, 8))
```

gives the following output.

```
#> [1] 5
```

Here:

- `mean` is the function;
- `c(2, 4, 6, 8)` is the input;
- the function calculates the mean.

Functions are generally followed by parentheses:

```
function_name()
```

The information that you give to a function is called an **argument**.

For example:

```
round(3.14159, digits = 2)
#> [1] 3.14
```

Here:

- `round()` is the function;

- 3.14159 is the value being rounded;
- `digits = 2` tells R how many decimal places to use.

2.4.3 Objects

Objects are another fundamental idea in R.

An object is something that R can store and work with.

For example:

```
age <- 21
```

Here we have created an object called `age`.

We can then ask R to show us its value:

```
age  
#> [1] 21
```

We can also use it in another calculation:

```
age + 1  
#> [1] 22
```

2.4.3.1 Assignment

The `<-` symbol is commonly used to assign a value to an object.

For example:

```
my_number <- 10
```

This means store the value 10 in an object called `my_number`.

Objects can contain different types of information. They do not have to contain just single numbers.

For example, an object can contain text:

```
student_name <- "Alex"
```

Or several numbers:

```
test_scores <- c(65, 72, 81, 69, 88)
```

For now, the important thing is simply to understand that R allows you to **store information in objects and then use those objects later**.

2.4.3.2 Naming objects

It is a good idea to give objects sensible names. When naming objects, best practices are to:

1. Be informative about that the object represents
2. Do not use spaces
3. If it has multiple words in the name, separate these by underscores or capitalise each word.
4. Do not include dots or other symbols

Good examples are:

```
age <- 21
average_score <- 75
TestScores <- c(65, 72, 81, 69, 88)
```

Compared with bad names:

```
x <- 21
average score <- 75
TEST.Scores! <- c(65, 72, 81, 69, 88)
```

The last two objects `average score` and `TEST.Scores!` are so bad that they will give you the following error!

```
#> Error in parse(text = input): <text>:1:9: unexpected symbol
#> 1: average score
#>      ^
```

2.4.4 Comments

You can add comments to your R code using the `#` symbol.

Anything after `#` on that line will be treated as a comment rather than R code and won't be executed when you run that line or script.

For example:

```
# Calculate the mean of the scores
scores <- c(1,2,3,4,5)
mean(scores)
```

Comments are useful for explaining what your code is doing.

You can also use comments to organise a script:

```
# -----
# Calculate summary statistics
# -----
mean(scores)
median(scores)
sd(scores)
```

Good habit: Write comments that help you — and someone else — understand what your code is doing.

Keeping your code organised. As your R scripts become longer, organisation becomes increasingly important.

You can use comments and headings to divide your code into sections.

For example:

```
# Load packages

library(ggplot2)

# Import data

data <- read.csv("my_data.csv")

# Explore data

head(data)
summary(data)

# Analysis

mean(data$score)

# Visualisation

plot(data$score)
```

This makes your script much easier to read.

2.4.5 Errors

Help! I've got an Error !

Getting errors, warnings and breaking your code are completely normal when programming - not just when you are learning it. Errors are not a sign that you are bad at coding. They are a very normal and it is likely you will see them frequently.

For example, if you type:

```
mean("This is not a number")
```

R will complain because it doesn't know how to take the mean of something that isn't numeric.

You may see an error message such as:

```
Warning message:
In mean.default("this is not a number") :
```

```
argument is not numeric or logical: returning NA
```

The error message is actually useful. It is R trying to tell you what went wrong.

:information_source: **When something goes wrong:**

1. Do not panic
2. Read the error message
3. Look at the line of code that caused the problem
4. Check your spelling
5. Check your brackets
6. Check your quotation marks
7. Check whether you have created the objects you are using
8. Search the error message if you are unsure

:+1: It is just as important to learn how to debug your code as it is learning to write code.

When stuck for coding solutions, experienced R users regularly:

1. Search Online

Often, a quick Google (or your preferred search engine) will give you the syntax you need to perform/fix what you want in R.

2. Look at documentation

Some of the more frequently used packages such as ggplot2 (a powerful packages for creating graphs) have very thorough documentation that can be very helpful in learning how to use it.

3. Use R help pages

R has a built-in help system. For example, if you want to find out more about the `mean()` function, you can type:

```
?mean
```

The Help pane in RStudio will then display information about the function. See **Getting to Know Rstudio** to find out more information about the Help pane.

R help pages can initially look intimidating. You will usually see information about:

- What the function does.
- How to use it.
- The value it returns.
- Examples of it in use.

The **Examples** section is particularly useful when you are learning.

You can often copy an example, run it yourself and then modify it.

4. Ask peers

In industry, academia and any other workplace it is very common to ask peers and colleagues for help with coding. Often, a solution to your problem will already exist and they may have seen it somewhere before. This is especially useful when your code is giving you errors - an extra pair of eyes always helps! In some cases, you often spot the error as your explaining your out loud to someone else.

5. Using AI

It's fairly commonplace now to use AI to debug or write code for you. Whilst this can be incredible useful it should be used with caution. Generative AI has a habit of producing over complex code that, whilst looking shiny and professional, is very hard to understand, modify and fix if you don't have a good grasp on the underlying language.

Understanding the concepts of coding are fundamental in providing solutions to problems and performing novel research. The long-term benefit of learning, reading and writing code yourself far outweigh the short term benefit of generating a sophisticated analysis with no development of your understanding.

Not only this, for some university assessments, the use of AI may not be tolerated at all, so it is good to check that you are using AI in a way that is honest, fair and responsible. Thorough guidance on the use of AI at The University of Sheffield can be found on the StudySkills@Sheffield webpage.

2.5 MASH support

There are lots of ways to learn R, and you do not have to learn it alone. Here at MASH, we offer friendly and specialist support from workshops to 1:1 sessions to suit your needs. Here are the main ways we can support you.

2.5.1 MASH workshops

MASH runs workshops covering statistics, mathematics and software skills.

R workshops are a great way to learn alongside other students and ask questions as you go.

We offer R workshops for ultra-beginners. These are designed for people who want to learn R from the very basics, without any prior knowledge - excellent if you have never coded before.

We also have more bespoke R workshops such as

- MASH workshop: Using R for data visualisation
- MASH workshop: Using R for hypothesis testing
- MASH workshop: Using R for regression

You can find and book workshops on the MASH workshops booking calendar

2.5.2 MASH online resources

The MASH First steps in R resources are designed to help you get started from the beginning, providing videos and help pages for getting started with R.

2.5.3 MASH 1-1 support

If you are stuck, you can also book a 1-1 appointment with MASH.

A 1:1 can be particularly useful if you have a specific question about:

- A research problem
- How to implement a statistical method
- R code that is not working
- Interpreting your output

You can book a 1-1 appointment on the MASH statistics 1-1 appointment booking page

2.6 Where to go next

Once you have got through the basics, there are lots of other resources you can use to continue learning.

2.6.1 YaRrr! The Pirate's Guide to R

YaRrr! The Pirate's Guide to R is a clear, free online textbook.

It provides an accessible introduction to R and includes examples that you can work through yourself.

2.6.2 R for Data Science

R for Data Science by Hadley Wickham is a great guide for importing, wrangling, manipulating and visualising data.

2.6.3 Learn R by example

Learn R by example is an easily to follow website organised around examples.

It covers topics ranging from basic R syntax through data structures, functions, data import, graphics and more.

2.6.4 Swirl

Swirl is a package for learning R interactively within RStudio.

Instead of reading instructions on a webpage and then switching between your browser and RStudio, Swirl gives you interactive lessons directly in R.

Begin by installing the `swirl` package.

Type the following command into the **Console**:

```
install.packages("swirl")
```

Wait for the installation to finish.

Once the installation is complete, load the package:

```
library("swirl")
```

Then start Swirl by typing:

```
swirl()
```

Swirl will begin interacting with you. Simply read the instructions on the screen and enter your responses.

2.7 Step-by-step process for learning R

Important: There is no single “correct” way to learn R.

However, a useful approach is:

Step 1 — Get R and RStudio installed

Make sure both are installed and that you can open RStudio.

Start with the MASH First steps in R resources if you need help.

Step 2 — Learn the basic language

Get comfortable with:

```
# Objects
my_number <- 10

# Vectors
my_numbers <- c(10, 20, 30, 40)

# Functions
mean(my_numbers)

# Indexing
my_numbers[1]
```

Executing `my_numbers[1]` will give the value 10, the number with *index* 1 (i.e. the 1st number) of the vector `my_numbers`.

Knowing how to index in R is incredibly useful for removing, adding and selecting specific rows of your data, as well as other things. It is well worth spending time to learn about it.

Note: If you have used other programming languages before, you will be aware that 0 is usually the first index of a vector. This is different in R. Indexing starts from 1.

Step 3 — Practise

Do not just read R code.

- Type it
- Change it
- Break it
- Fix it

Try to predict what will happen before you run it.

Step 4 — Work with R data sets

Once you understand the basics, start working with data. R has an amazing library of built-in real world datasets. These are nice clean examples of data that are frequently used to demonstrate visualisation and data analysis.

To access the data sets you need to install the `datasets` package using the following commands.

```
install.packages("datasets")
library(datasets)
```

You can then run the command

```
data()
```

to view a list of the datasets available. To load a dataset you simply type the dataset. For example, for the `iris` dataset you simply run:

```
head(iris)
#>   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#> 1         5.1         3.5         1.4         0.2  setosa
#> 2         4.9         3.0         1.4         0.2  setosa
#> 3         4.7         3.2         1.3         0.2  setosa
#> 4         4.6         3.1         1.5         0.2  setosa
#> 5         5.0         3.6         1.4         0.2  setosa
#> 6         5.4         3.9         1.7         0.4  setosa
```

Note: By putting `iris` in the function `head()`, R only shows us the first few rows of the data.

Important concepts to learn with the example data are:

- Importing data
- Cleaning data
- Selecting variables
- Adding new variables

- Plotting graphs
- Statistical analysis

Step 5 — Build something

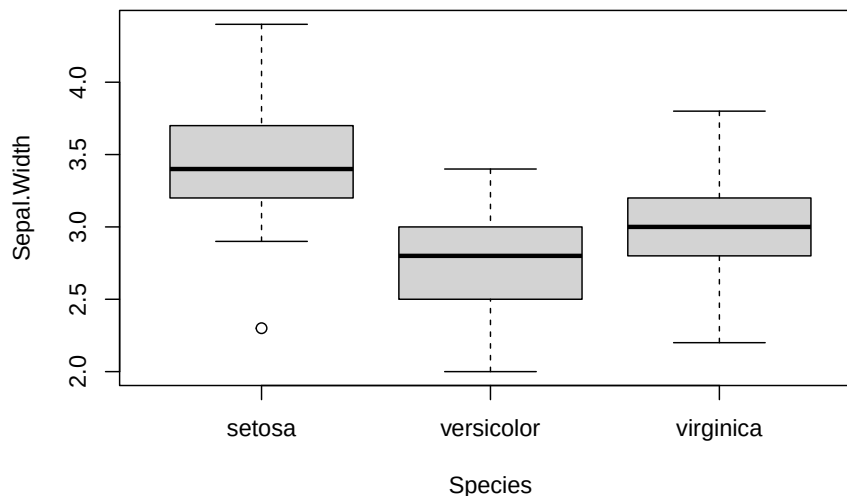
The best way to learn R is to use it. Try using R for something that you actually need to do.

For example:

“I have a dataset. Can I use R to calculate some summary statistics and make a graph?”

Maybe we wanted to create a graph showing the differences in sepal width between species in the `iris` dataset. We can make a quick plot using the following code.

```
boxplot(  
  Sepal.Width ~ Species,  
  data = iris  
)
```

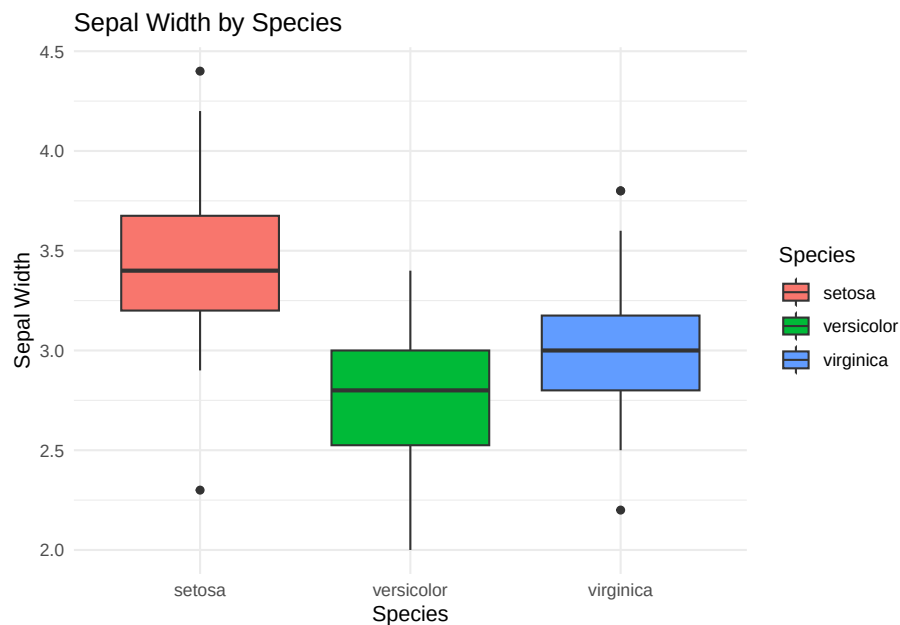
**Step 6** - Develop it

Once you have a minimal viable product of what you want to create, now it is time to develop your code. This could mean, making plots look better, making analysis more sophisticated, or simply organising your code better for yourself or another reader.

For example, we might improve our code above by using the `ggplot2` package and adding in some comments for the user.

```
# Load in ggplot packages
library(ggplot2)

# Plot a boxplot of sepal widths coloured by iris species
ggplot(iris, aes(x = Species, y = Sepal.Width, fill = Species)) +
  geom_boxplot() +
  labs(
    title = "Sepal Width by Species",
    x = "Species",
    y = "Sepal Width"
  ) +
  theme_minimal()
```



2.8 Final advice

When learning R, you may sometimes feel that everyone else understands it except you.

That's not true.

Even experienced R users regularly search for functions, look at documentation, make mistakes and spend time figuring out why something isn't working.

The difference is that, with practice, you gradually become better at solving those problems.

You do not need to become an expert before you can use R.

You just need to keep building your knowledge.

Start small. Practise often. Don't be afraid of errors. Ask for help.

2.9 Your R learning checklist

Use this checklist to keep track of your progress.

- ☐ I understand what R is.
- ☐ I understand the difference between R and RStudio.
- ☐ I have installed R.
- ☐ I have installed RStudio.
- ☐ I can open RStudio.
- ☐ I know where to find the MASH First steps in R resources page.
- ☐ I know what the Console is.
- ☐ I know what an R script is.
- ☐ I understand the difference between the Console and a script.
- ☐ I know what a command is.
- ☐ I know what a function is.
- ☐ I know what an argument is.
- ☐ I can create an object.
- ☐ I can create a vector.
- ☐ I can call a function.
- ☐ I know what a package is.
- ☐ I know the difference between installing and loading a package.
- ☐ I know how to find help for a function.
- ☐ I understand what a working directory is.
- ☐ I know how to save an R script.
- ☐ I know how to book a MASH 1:1 appointment.
- ☐ I know how to book a MASH R workshop

2.10 Glossary

Term	Brief description
R	A programming language used for statistics, data analysis and data visualisation.

Term	Brief description
RStudio	An Integrated Development Environment (IDE) that provides an interface for working with R.
Command	An instruction given to R to perform an action.
Function	A piece of code designed to perform a particular task.
Argument	Information supplied to a function that tells it what to work with or how to behave.
Object	Something that R can store and work with, such as a number, text or vector.
Assignment	The process of storing a value in an object, commonly using <code><-</code> .
Package	A collection of additional functions, data and resources that extends R's capabilities.
Script	A saved file containing R code, usually with the <code>.R</code> file extension.
Console	The RStudio pane where R commands are entered and executed.
Source	The RStudio pane where you write, edit and save R scripts.
Environment	The RStudio pane showing objects currently stored in your R session.
History	An RStudio tab showing commands previously entered during your session.
Working directory	The folder R uses as its default location for reading and writing files.
Comment	Text beginning with <code>#</code> that R ignores when executing code.
Debugging	The process of finding and fixing problems in code.
Error	A message from R indicating that something has gone wrong when running code.
Warning	A message from R indicating that something may not have worked as expected.

Chapter 3

Downloading R and RStudio

3.1 Introduction

This is the first document in our “First Steps in *R*” series of resources.

R is a programming language specifically designed for doing statistics. The *R* software can be downloaded on its own but it is more usual to download both *R* and *RStudio*. *RStudio* is a more convenient way of interacting *R*. The *R software* must be installed before *RStudio* will work.

3.2 Downloading *R* for Windows (do this first, before downloading *RStudio*)

R can be downloaded from CRAN (the Comprehensive R Archive Network).

Click “**Download R-4.4.1 for Windows**”.

Figure 1: The link to click to download R for Windows.

When the file has downloaded, open it just as you would when installing any other new program.

We would recommend you agree with the default settings and perhaps consider clicking the boxes to add a desktop shortcut and a *Quick Launch* shortcut.

3.3 Downloading *R* for other operating systems

Go to CRAN (the Comprehensive R Archive Network). Select the appropriate operating system from the options given then download the latest release from the following page.

Figure 2: The links to click to download R for other operating systems.

3.4 Downloading *RStudio* (you will need to download *R* first to enable *RStudio* to work)

Go to R Studio.

Scroll down a little and choose the free version of *RStudio* and click “**Download.**”

Figure 3: The button to click to download RStudio.

Click “**Download *RStudio* for Windows**” (or select your operating system from the list a little further down).

Again, open the file just as you would when installing any other new program and select the default options.

Chapter 4

First interactions with R and RStudio

4.1 Introduction

This document is part of our “First Steps in **R**” resources. It is assumed that the reader has downloaded **R** and **RStudio**. No other pre-knowledge is required.

Most of the time we are used to interacting with a computer through a Graphical User Interface (GUI) (i.e. with folders, files, windows etc). In **R** and **RStudio**, we type instructions (code) instead. The window the instructions are typed into is called the console. Of course, we must type our instructions in a way the computer will understand - this is why it’s called a programming *language*. This is the way computers were operated before the development of GUIs. **R** is a programming language which has been developed specifically for doing statistics. **R** and **RStudio** are pieces of software which allow the computer to read the code we type and execute it (i.e. follow the instructions).

4.2 Opening **R**

If you open the **R** software you will get a window in which you can type code. This is the console. It is possible to use the **R** package to do statistics with **R** but **RStudio** is much more friendly to use. **RStudio** is called an Integrated Development Environment (IDE) for **R**. Note that for **RStudio** to work, it requires that **R** is also installed. Because of its ease of use, once you’ve used **RStudio**, you will probably find you never open **R** again! All of the materials in this series use **RStudio** for this reason.

4.3 First Opening *RStudio*

When you open *RStudio* for the first time there will be three windows. The one on the left is the console. You can type code directly in here and *RStudio* will follow your instructions as soon as you press return. One thing *R* can do is simple calculations like a calculator. You can experiment by typing calculations in the console. If you type $4+6$ and press return, *R* will give you the answer 10.

Figure 1: RStudio when it is opened for the first time.

If you'd like to carry out other calculations, here are the ways you must enter them into *RStudio* for some simple operations.

Arithmetic Operation	R Code
$4 + 6$	<code>4+6</code>
$6 - 4$	<code>6-4</code>
4×6	<code>4*6</code>
$4 \div 6$	<code>4/6</code>
4^6	<code>4^6</code>
e^4	<code>exp(4)</code>
$\log_{10}(4)$	<code>log10(4)</code>
$\ln(4)$	<code>log(4)</code>

Note that you cannot go back and delete text you have put in the console. If you've never done any coding at first this may seem strange but you'll get used to it quickly.

4.4 Inputting Code Using a Script

A Script is the name for some code that has been written and which we want *R* to execute for us. You can write a script, edit it, save it then open it in *RStudio* and run it whenever you like. *RStudio* will let you edit your script until you're happy with it and then run all or just some of it by highlighting and clicking run. If you choose "Save As..." in the file menu (as you would when saving, say, a *Word* document) the script is what gets saved. To create a **Script file** for the first time, in *RStudio*, click "**File**" then select "**New File**" and "**R Script**."

Figure 2: How to create a Script file in RStudio for the first time.

A fourth window opens up on the top left in *RStudio*. Try typing a few calculations in here. You will see that *RStudio* will let you type without carrying out any of the calculations. See what happens if you highlight the text and click "**Run**."

Figure 3: The fourth window, for scripts, in RStudio.

You should find that all the calculations are carried out at once in the console like so:

Figure 4: The calculations are carried out in the console in RStudio all at once when the script is run.

4.5 The Environment

As we write code, we will create “*objects*”. The word has a technical use in *R* programming which we will deal with in future worksheets. For now we will think of an object as being similar to a variable in algebra - i.e. a letter to which we can assign a value.

We can tell *R* that we are creating a new object using the “*assignment operator*”:

```
<-
```

You can think of this as an arrow. The name of the new object should go on the left and the definition goes on the right. So the code

```
x <- 3
```

would create a new object called *x* which is defined to have the value 3. If we then type

```
x+4
```

We should find that *R* calculates $3 + 4$ and returns the answer 7.

Type

```
x <- 3
```

in the console then look in the top right window. You should find that it looks like this.

Figure 5: The top right window (the “Environment”) in RStudio, where *x* is defined to have the value 3.

(Note: You can use $=$ as an assignment operator as well as $<-$. However, there is a technical difference in the way *R* reads $=$. In most situations, either $=$ or $<-$ can be used interchangeably but it is a good practice to stick to $<-$ when defining a new object.)

When we are writing code we may want to define many new objects. These objects then exist in the *R* “Environment”. As we create new objects, the top right window keeps a record of them for us. This helps us keep track and can help us ensure that we do not create two objects with the same name.

We can ask *R* to give us the same information by typing `ls()` in the console. “ls” is short for list - we are asking *R* to list all the objects in the environment.

You can remove an object from the environment by typing

```
rm()
```

with the name of the object in the brackets.

For example,

```
rm(x)
```

would remove our variable `x` from the environment.

You can delete all objects from the environment by clicking the broom icon in the top ribbon of the “Environment” window:

Figure 6: How to delete all objects from the environment in RStudio.

Alternatively, you can type the following command in the console:

```
rm(list=ls())
```

(Note: we don’t need to worry about the details of this command at the moment but in case you’re interested: “rm” tells **R** to remove the thing in the brackets. The rest of the code tells **R** that it is going to remove an object which is a “list” and that list is called `ls()`. A “list” is a category of object in **R**. `ls()` is specifically the list of everything in the environment.)

4.6 The Bottom Right Window

There are several tabs in the window in the bottom right. For the moment we will only worry about two of them: “Plots” and “Help”. If we ask **RStudio** to draw a graph or chart, it will appear in this window under this tab. If we ask **RStudio** for help - which we will often do when learning how to use a new piece of code - the answer will appear under the “Help” tab.

4.7 Exercise

1. In the console, create an object called `y` and give it the value 15.
2. Create another object called `z` and give it the value 5.
3. Calculate `y` divided by `z`.
4. Clear the environment and repeat the above by writing a script to do all three stages and running it.
5. Change the value of `y` in your script to 20 and run it again.

4.7.1 Solutions

Solutions

1. `> y <- 15`

2. `> z <- 5`

`> y`
`[1] 15`

`> z`
`[1] 5`

3. `> y/z`
`[1] 3`

4. Script:

`y <- 20`

`z <- 5`

`y`

`z`

`y/z`

And here it is run:

`> y <- 20`

`> z <- 5`

`> y`

`[1] 20`

`> z`

`[1] 5`

`> y/z`

`[1] 4`

Chapter 5

Vectors

5.1 Introduction

This document is part of our “First Steps in *R*” resources. It is assumed that the reader has downloaded *R* and *RStudio* and is familiar with the idea of typing commands into the console. If you would like a reminder of these ideas, resources can be found on the MASH website.

John Chambers, one of the creators of the *R* programming language, once said “To understand computations in *R*, two slogans are helpful: 1. Everything that exists is an object. 2. Everything that happens is a function call.” Objects in *R* come in various categories such as Vectors, Matrices, Data Frames and Lists. The simplest of these is vectors.

The word vector is used in different ways in different contexts. In *R* we can think of a vector “as a list of things in order”, similar to the idea of a vector in mathematics.

5.2 Defining a Numerical Vector

To define a new object in *R* we must give it a name. The name must be a continuous string of characters with no spaces. For example, if we wanted to define an object called “New Vector” sensible names would be

```
NewVector new_vector new.vector
```

Once we have chosen a name for our new object, we need to assign it a value. We do this with the assignment operator

```
<-
```

So if we type

```
new_vector <-
```

we have so far told **R** that we are going to define a new object called `new_vector`. What we type next will be the definition of the object. To define a vector we need to use the letter `c` for “combine”. So the command

```
new_vector <- c(2,4,6,8)
```

would be read by **R** as “combine the numbers 2, 4, 6 and 8 into a vector and give them the label `new_vector`”. Note that the elements of a vector must be separated by commas as shown.

When typing code it is important to be precise. A single character out of place might make the code unreadable for the computer. However, **R** will ignore spaces (unless they are in the middle of the name of the object). So **R** would read all of the following identically:

```
new_vector <- c(2,4,6,8) new_vector <- c( 2 , 4 , 6 , 8 ) new_vector <-c(2,4 ,
6 , 8)
```

Define an object called `new_vector` as above and look what happened in the “Environment” window (top right). You should see that the vector has appeared in the environment as follows:

Figure 1: The top right window (the “Environment”) in RStudio, where `new_vector` is shown to be stored as a numerical vector.

“num” means that the vector has been stored by **R** as a numerical vector and **R** is reading the elements of the vector as numbers; `1:4` means that **R** has labelled the elements of the vector as first, second, third and fourth and the final part lists the elements of the vector.

We can now ask **R** to show us the vector by typing in the console window:

```
new_vector
```

R will then display our vector like so:

Figure 2: `new_vector` is shown in the console in RStudio.

The `[1]` means that the first entry on that line is the first element of the vector.

We can also carry out calculations with our vector. Type the following:

```
new_vector +2
```

and **R** will add 2 to each element of the vector like so:

Figure 3: 2 is added to each element of `new_vector` in the console in RStudio.

5.3 Vectors using “.”

Type the following into the console

```
vec2 <- 1:100 vec2
```

The first line defines a new object called `vec2` and the second line asks **R** to display that object. You should see that `vec2` is a vector which consists of the whole numbers from 1 to 100. Looking in the environment will confirm that **R** has stored `vec2` as a numerical vector:

Figure 4: The “Environment” window in RStudio where `vec2` is shown to be stored as a numerical vector.

“:” is a shortcut to create vectors like this. Experiment by typing the following commands into the console

```
2:16 5.6:14 3.9:10
```

5.4 Character Vectors

A character vector is similar to a numerical vector but **R** will interpret the elements as words rather than numbers. This might be helpful if, for example, we have some data about different people and we want to include a list of their names. We define a character vector just like a numerical vector but we must put each element of the vector in inverted commas in the definition. For example:

```
people <- c(“Ali”, “Bertha”, “Charlie”)
```

defines a character vector called `people` which would store the names Ali, Bertha and Charlie. If you type this into the console you will see the new object appear in the environment like so:

Figure 5: The top right window (the “Environment”) in RStudio where `people` is shown to be stored as a character vector.

“chr” shows that **R** is interpreting `people` as a character vector. Note that we did not need to tell **R** what sort of vector we were defining. Putting the elements in inverted commas was enough for **R** to know that our vector was a character vector.

The elements of a character vector are called **strings**.

In **R** “ can often be replaced by ’ without noticing the difference. The usual convention is to use “.

(For completeness: if, for some reason, you want to define a string which contains the character “, you should use ’ at the beginning and the end of the definition.)

You will sometimes see vectors referred to as **atomic vectors**, which means that they can only contain one type of data. For example, a vector can contain either numeric or character values but not both in the same vector. If we try to define a vector with some numbers and some character strings, **R** will interpret the numbers as character strings.